Questions 6–9 in the Data Structure Lab Cycle are the linked-list programs shown in the uploaded lab sheet.

DATA STRUCTURES LAB – QUESTIONS 6 TO 9

Q6. IMPLEMENT A SINGLY LINKED LIST

Algorithm:

1. Start.
2. Define a node containing data and next pointer.
3. Create functions:
   a) Create List
   b) Insert Before a Given Value
   c) Insert After a Given Value
   d) Delete a Given Value
   e) Display List
4. Read user choice.
5. Perform the selected operation.
6. Repeat until Exit.
7. Stop.

Program:

```
#include <stdio.h>

#include <stdlib.h>


struct node {

  int data;

  struct node *next;

};


struct node *head = NULL;


void create(int n) {

  struct node *temp, *newnode;

  int i, val;


  for(i = 0; i < n; i++) {
```

```
      scanf("%d", &val);

      newnode = (struct node*)malloc(sizeof(struct node));
      newnode->data = val;
      newnode->next = NULL;

      if(head == NULL) {
         head = newnode;
         temp = newnode;
      } else {
         temp->next = newnode;
         temp = newnode;
      }
   }
}

void insertAfter(int key, int val) {
   struct node *ptr = head;

   while(ptr != NULL && ptr->data != key)
      ptr = ptr->next;

   if(ptr != NULL) {
      struct node *newnode =
         (struct node*)malloc(sizeof(struct node));

      newnode->data = val;
      newnode->next = ptr->next;
      ptr->next = newnode;
   }
}
```

```
void insertBefore(int key, int val) {
   struct node *ptr = head, *prev = NULL;

   while(ptr != NULL && ptr->data != key) {
      prev = ptr;
      ptr = ptr->next;
   }

   if(ptr != NULL) {
      struct node *newnode =
         (struct node*)malloc(sizeof(struct node));

      newnode->data = val;

      if(prev == NULL) {
         newnode->next = head;
         head = newnode;
      } else {
         prev->next = newnode;
         newnode->next = ptr;
      }
   }
}

void deleteNode(int key) {
   struct node *ptr = head, *prev = NULL;

   while(ptr != NULL && ptr->data != key) {
      prev = ptr;
      ptr = ptr->next;
```

```
    }

    if(ptr != NULL) {
        if(prev == NULL)
            head = ptr->next;
        else
            prev->next = ptr->next;

        free(ptr);
    }
}

void display() {
    struct node *ptr = head;

    while(ptr != NULL) {
        printf("%d -> ", ptr->data);
        ptr = ptr->next;
    }
    printf("NULL\n");
}

int main() {
    int n, ch, key, val;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    create(n);

    do {
```

```
    printf("\n1.Insert Before\n2.Insert After\n3.Delete\n4.Display\n5.Exit\n");
    scanf("%d", &ch);

    switch(ch) {
      case 1:
        scanf("%d%d", &key, &val);
        insertBefore(key, val);
        break;

      case 2:
        scanf("%d%d", &key, &val);
        insertAfter(key, val);
        break;

      case 3:
        scanf("%d", &key);
        deleteNode(key);
        break;

      case 4:
        display();
        break;
    }
  } while(ch != 5);

  return 0;
}
```

Q7. ADDITION OF POLYNOMIALS USING SINGLY LINKED LIST

Algorithm:

1. Create linked lists for two polynomials.

2. Each node contains coefficient and exponent.
3. Compare exponents of both polynomials.
4. If exponents are equal, add coefficients.
5. Otherwise copy the term with larger exponent.
6. Continue until both lists are processed.
7. Display the resultant polynomial.
8. Stop.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct poly {
  int coeff;
  int exp;
  struct poly *next;
};

void insert(struct poly **head, int c, int e) {
  struct poly *newnode =
    (struct poly*)malloc(sizeof(struct poly));

  newnode->coeff = c;
  newnode->exp = e;
  newnode->next = NULL;

  if(*head == NULL)
    *head = newnode;
  else {
    struct poly *temp = *head;
    while(temp->next)
      temp = temp->next;
```

```
        temp->next = newnode;
    }
}

struct poly* add(struct poly *p1, struct poly *p2) {
    struct poly *res = NULL;

    while(p1 && p2) {
        if(p1->exp == p2->exp) {
            insert(&res, p1->coeff + p2->coeff, p1->exp);
            p1 = p1->next;
            p2 = p2->next;
        }
        else if(p1->exp > p2->exp) {
            insert(&res, p1->coeff, p1->exp);
            p1 = p1->next;
        }
        else {
            insert(&res, p2->coeff, p2->exp);
            p2 = p2->next;
        }
    }

    while(p1) {
        insert(&res, p1->coeff, p1->exp);
        p1 = p1->next;
    }

    while(p2) {
        insert(&res, p2->coeff, p2->exp);
        p2 = p2->next;
```

```
    }

    return res;
}

void display(struct poly *head) {
    while(head) {
        printf("%dx^%d", head->coeff, head->exp);
        head = head->next;
        if(head)
            printf(" + ");
    }
    printf("\n");
}

int main() {
    struct poly *p1 = NULL, *p2 = NULL, *res;

    insert(&p1, 5, 2);
    insert(&p1, 4, 1);

    insert(&p2, 3, 2);
    insert(&p2, 2, 0);

    res = add(p1, p2);

    printf("Result: ");
    display(res);

    return 0;
}
```

Q8. IMPLEMENT A DOUBLY LINKED LIST

Algorithm:

1. Define a node with data, previous pointer and next pointer.
2. Create nodes and link them.
3. Insert nodes as required.
4. Delete nodes if needed.
5. Traverse forward and display.
6. Traverse backward and display.
7. Stop.

Program:

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev;
   struct node *next;
};

struct node *head = NULL;

void insertEnd(int val) {
   struct node *newnode =
      (struct node*)malloc(sizeof(struct node));

   newnode->data = val;
   newnode->next = NULL;

   if(head == NULL) {
      newnode->prev = NULL;
```

```
    head = newnode;
    return;
  }

  struct node *temp = head;
  while(temp->next)
    temp = temp->next;

  temp->next = newnode;
  newnode->prev = temp;
}

void displayForward() {
  struct node *temp = head;

  while(temp) {
    printf("%d ", temp->data);
    temp = temp->next;
  }
  printf("\n");
}

void displayBackward() {
  struct node *temp = head;

  while(temp->next)
    temp = temp->next;

  while(temp) {
    printf("%d ", temp->data);
    temp = temp->prev;
```

```
    }
    printf("\n");
}

int main() {
    insertEnd(10);
    insertEnd(20);
    insertEnd(30);

    printf("Forward: ");
    displayForward();

    printf("Backward: ");
    displayBackward();

    return 0;
}
```

Q9. BROWSER HISTORY USING DOUBLY LINKED LIST

Algorithm:

1. Create a doubly linked list.
2. Each node stores a webpage URL.
3. Current pointer indicates current page.
4. Visit:
   a) Create new node.
   b) Attach after current page.
   c) Move current to new page.
5. Back:
   a) Move current to previous node.
6. Forward:
   a) Move current to next node.
7. Display Current Page.
8. Display Entire History.

9. Stop.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct page {
  char url[100];
  struct page *prev;
  struct page *next;
};

struct page *current = NULL;

void visit(char url[]) {
  struct page *newnode =
    (struct page*)malloc(sizeof(struct page));

  strcpy(newnode->url, url);
  newnode->next = NULL;

  if(current == NULL) {
    newnode->prev = NULL;
    current = newnode;
    return;
  }

  current->next = newnode;
  newnode->prev = current;
  current = newnode;
}
```

```
void back() {
   if(current && current->prev)
      current = current->prev;
}

void forward() {
    if(current && current->next)
      current = current->next;
}

void displayCurrent() {
   if(current)
      printf("Current Page: %s\n", current->url);
}

void displayHistory() {
   struct page *temp = current;

   while(temp && temp->prev)
      temp = temp->prev;

   printf("History:\n");

   while(temp) {
      printf("%s\n", temp->url);
      temp = temp->next;
   }
}

int main() {
```

```
    visit("google.com");
    visit("youtube.com");
    visit("github.com");

    displayCurrent();

    back();
    displayCurrent();

    forward();
    displayCurrent();

    displayHistory();

    return 0;
}
```

You can copy this into a Word document/PDF and upload it to Google Classroom for attendance.